

Z-Order Transformer for Feed-Forward Gaussian Splatting

Can Wang¹ Lei Liu¹ Wei Jiang² Dong Xu^{1†}
¹The University of Hong Kong ²Futurewei Technologies Inc.



Figure 1. We propose a Z-order transformer that reconstructs 3D Gaussians from arbitrary multi-view captures for novel view synthesis. Compared with previous methods, our approach achieves more effective and efficient 3D Gaussian Splatting (3DGS) reconstruction while requiring fewer Gaussian primitives.

1

Abstract

Recent advances in 3D Gaussian Splatting (3DGS) have enabled significant progress in photorealistic novel view synthesis. However, traditional 3DGS relies on a slow, iterative optimization process, which limits its use in scenarios demanding real-time results. To overcome this bottleneck, recent feed-forward methods aim to predict Gaussian attributes directly from images, but they often struggle with the redundancy of Gaussian primitives and rendering quality. In this work, we introduce a transformer-based architecture specifically designed for feed-forward Gaussian Splatting. Our key insight is that spatial and semantic relationships among Gaussians can be effectively captured through a sparse attention mechanism, enabled by a Z-order strategy that organizes the unstructured Gaussian set into a spatially coherent sequence. Furthermore, we incorporate this Z-order strategy to adaptively suppress redundancy while preserving critical structural details. This al-

lows the transformer to efficiently model context, compress Gaussian primitives, and predict Gaussian attributes in a single forward pass. Comprehensive experiments demonstrate that our method achieves fast and high-quality novel view synthesis with fewer Gaussian primitives.

1. Introduction

Recent advances in 3D Gaussian Splatting (3DGS) [9, 18, 40, 41, 48] have demonstrated remarkable progress in fast reconstruction, real-time rendering, and high-quality novel view synthesis by representing a scene as a collection of anisotropic Gaussian primitives optimized from multi-view images. However, the original 3DGS suffers from limited generalization capacity and remains time-consuming, as it relies on per-scene gradient-based optimization to fit the Gaussian parameters.

To address these limitations, recent studies have introduced feed-forward GS [1, 2, 8, 19, 37, 39, 46, 47], which replaces the costly per-scene optimization with a learned prediction framework. Instead of iteratively adjusting Gaus-

[†] Dong Xu is the corresponding author.

sian parameters through gradient descent, feed-forward GS employs a neural network to directly infer Gaussian attributes such as position, scale, opacity, and color from input images in a single forward pass.

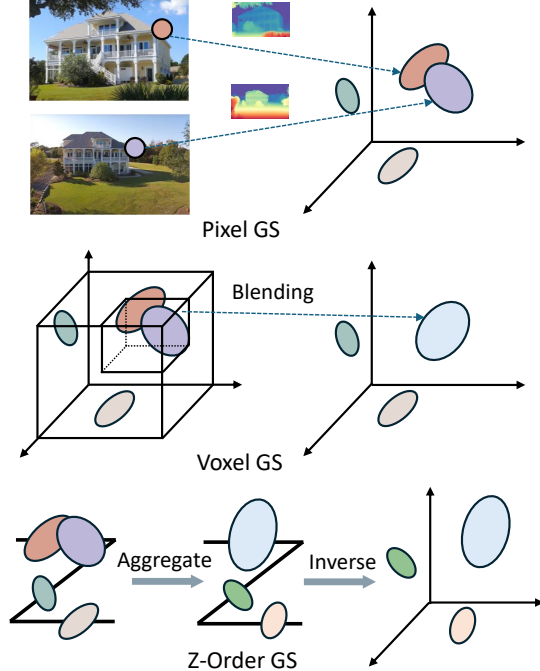


Figure 2. Gaussian representations in feed-forward GS.

A key component of current feed-forward GS is the pixel-level Gaussian representation [1, 2, 19, 37, 39, 46, 47], as illustrated in Fig. 2 (top). Each pixel in each input view is associated with one or multiple Gaussian primitives. The positions of these Gaussians are initialized by projecting the 2D pixel coordinates into 3D space through depth warping. This design allows Gaussian parameters to be directly predicted from dense image features through a lightweight Gaussian head, typically implemented with a CNN. However, even associating one Gaussian per pixel can lead to a large number of primitives, which increases memory consumption and rendering costs, especially for high-resolution or multi-view inputs. For example, two 512×512 input views produce over 50W Gaussian primitives.

As illustrated in Fig. 2 (middle), voxel-based GS [8, 27, 33] mitigates this problem by aggregating neighboring pixel-level Gaussians within each voxel cell via a blending strategy. While this method reduces redundancy, the discretization of 3D space inevitably introduces quantization errors that can blur fine details and sharp boundaries. Increasing the voxel resolution alleviates this problem but causes an exponential increase in memory usage and computational cost, as the number of voxels scales cubically with resolution. Furthermore, the fixed voxel grid is inefficient for representing irregular or sparse regions, leading to redundant empty cells and wasted computation.

We propose a Z-order [21] transformer to address the aforementioned issues. As illustrated in Fig. 2 (bottom), our method first encodes pixel-level GS into a Z-order GS representation, which forms a compact 2D sequence while preserving spatial locality. This design removes the need for a dense voxel grid, resulting in higher computational and memory efficiency. The Z-curve ordering further enables fast neighborhood access and consistent spatial grouping, facilitating efficient GS aggregation and scalable feed-forward inference. Moreover, we propose a sparse attention mechanism consisting of group attention and top-k attention to fully exploit the locality of the Z-order GS representation while avoiding the high computational cost of full attention. After aggregation, the Z-order GS representation can be mapped back to 3D space through the inverse Z-order transformation. Considering the availability of dense views during inference and the resulting efficiency concerns, we further propose a Z-order-based Maximum Coverage Viewpoint Selection algorithm to effectively reduce view redundancy and improve efficiency. Experimental results demonstrate that our Z-order transformer significantly improves efficiency and rendering quality while requiring fewer Gaussian primitives.

2. Related Work

Per-scene Optimized Gaussian Splatting. 3D Gaussian Splatting (3DGS) [9] is an efficient representation for novel view synthesis that models a 3D scene as a collection of anisotropic Gaussian primitives. By directly optimizing these primitives through differentiable rendering, 3DGS achieves high-quality, photorealistic results. It has motivated many subsequent works [3, 6, 10, 12, 40, 41, 44] to improve its efficiency and rendering quality. For example, MipSplatting [41] extends 3D Gaussian Splatting by introducing a multiscale, anti-aliased rendering formulation to address level-of-detail and aliasing issues, thereby enhancing rendering robustness and visual fidelity. However, these optimization-based 3DGS methods are tailored to individual 3D scenes, lacking real-time reconstruction capability and generalization ability. To address this limitation, we propose a feed-forward 3DGS framework.

Feed-forward Gaussian Splatting. Feed-forward 3DGS employs a learned prediction network to directly infer Gaussian attributes from input images in a single forward pass. At present, most feed-forward GS methods [1, 2, 19, 37, 39, 46, 47] predict pixel-wise Gaussians based on pixel-aware features. For instance, PixelSplat [1] and DepthSplat [37] predict 3D Gaussian primitives that are spatially aligned with pixel-level features from input images, achieving efficient and scalable novel view synthesis. However, pixel-level-based methods often generate an excessive number of primitives, leading to increased memory consumption and rendering costs. To overcome this limitation,

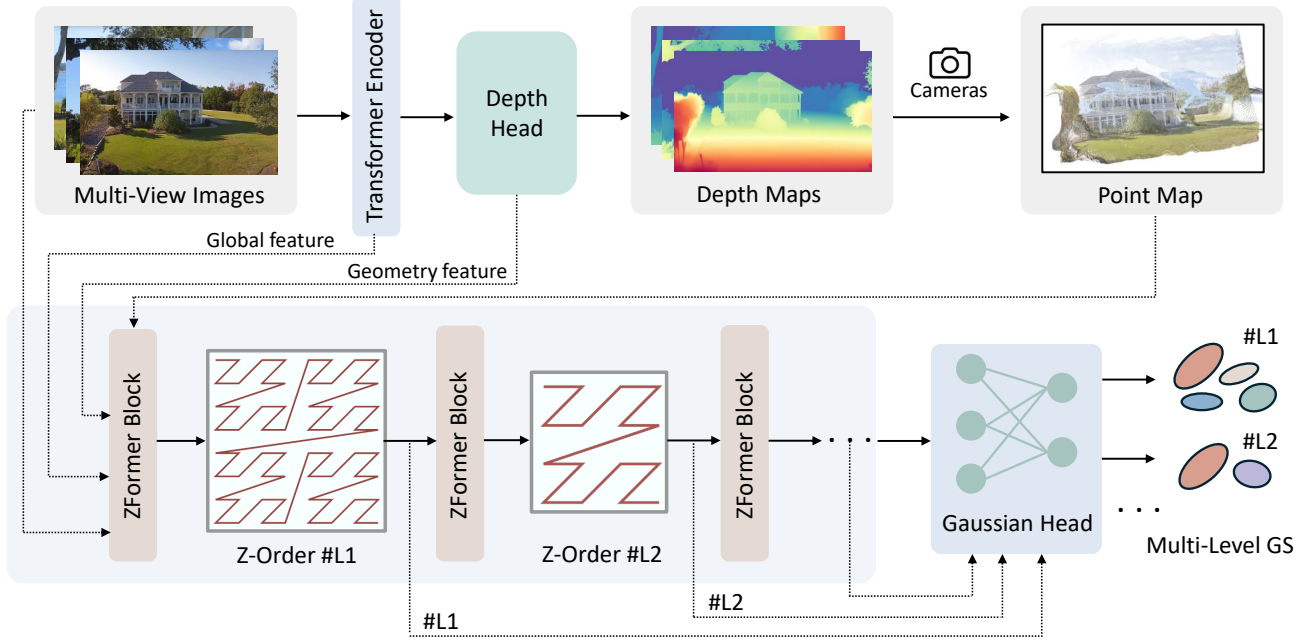


Figure 3. **Framework.** Given multi-view images, our method first utilizes a transformer encoder and a depth head to generate depth maps, which are then projected into a 3D point map based on the camera. Next, global features are extracted from the transformer encoder, and geometry features are derived from the depth head. These features, along with the pixel color and point map, are processed through our ZFormer blocks to generate Z-order-based Gaussian representations. Finally, the Gaussian representations are passed to the Gaussian head, which generates multi-level GS for further rendering.

voxel-based methods[8, 27, 33] have been proposed. For instance, AnySplat [8] adopts voxel-level techniques to optimize the processing of 3D Gaussian primitives through a differentiable voxelization module. However, the fixed voxel grid is inefficient for representing irregular or sparse regions, leading to numerous empty cells and redundant computations. To address this, our method introduces a Z-order-transformer that enables fast access to neighboring Gaussians and consistent spatial grouping, resulting in an efficient and high-quality feed-forward GS representation.

Z-order Strategy. Z-order [21] is a space-filling curve technique that converts multi-dimensional data into a low-dimensional sequence, with the goal of preserving spatial locality so that neighboring points in high-dimensional space remain adjacent in the lower-dimensional ordering. Benefiting from its ability to preserve spatial locality, Z-order has been extensively utilized in spatial databases to enable efficient indexing and range queries [11, 24, 25], in computer graphics for texture mapping and hierarchical level-of-detail control [15, 29], and in point cloud processing for serialization and fast neighborhood retrieval [4, 13, 32, 36]. For instance, Point Transformer V3 [36] presents a simplified and scalable 3D backbone utilizing Z-order-based point serialization to enable efficient neighborhood aggregation, broader receptive fields, and superior speed and memory efficiency. This motivates us to integrate the Z-order principle into 3DGS to enhance spatial coher-

ence and achieve a more efficient and robust scene representation.

3. Method

As shown in Fig. 3, we propose a Z-order transformer that performs feed-forward 3DGS reconstruction for novel view synthesis given a set of multi-view image captures. The structure of the following part is organized as follows: First, we provide an overview of the key concepts, including feed-forward 3D Gaussian prediction and the Z-order curve (Sec. 3.1). Next, we introduce the Z-order splat transformer (Sec. 3.2). We then present a multi-stage training strategy to train our framework (Sec. 3.3). Finally, we propose a Z-order-based maximum coverage viewpoint selection algorithm to reduce view redundancy and inference costs when dense captures are available (Sec. 3.4).

3.1. Preliminaries

Feed-forward 3D Gaussian Splatting. Given N views of a 3D scene $\mathbf{I} = \{I\}_{i=1}^N \in \mathbb{R}^{N \times 3 \times H \times W}$, feed-forward 3DGS refers to the direct prediction of Gaussian primitives via a neural network $\mathcal{F}$, as follows:

$$\{G_k : (\mu_k, \sigma_k, r_k, s_k, c_k)\}_{k=1}^K = \mathcal{F}(\{I\}_{i=1}^N) \quad (1)$$

Each Gaussian G_k is characterized by a set of parameters: $\mu_k \in \mathbb{R}^3$ represents the mean of the k^{th} Gaussian,

$\sigma_k \in \mathbb{R}^+$ is the opacity, $r_k \in \mathbb{R}^4$ is the orientation quaternion, $s_k \in \mathbb{R}^3$ is the anisotropic scale, $c_k \in \mathbb{R}^{27}$ is the color represented by 2nd-order spherical harmonics (SH) coefficients. The key challenge of feed-forward 3DGS is to design a function $\mathcal{F}$ that is highly efficient and capable of predicting high-fidelity Gaussian primitives using as few Gaussian points as possible. Our work addresses this challenge with a novel Z-order transformer architecture that efficiently and effectively predicts compact yet high-quality Gaussian representations.

Z-order Curve. The Z-order curve, also known as the Morton curve or Morton order, is a space-filling curve that maps multi-dimensional data onto a one-dimensional linear ordering while preserving spatial locality [21, 36, 43]. Let a 3D point be given as $P = (x, y, z)$. Each coordinate is expressed in binary using at most $depth = d$ bits:

$$x = \sum_{i=0}^{d-1} x_i 2^i, \quad y = \sum_{i=0}^{d-1} y_i 2^i, \quad z = \sum_{i=0}^{d-1} z_i 2^i \quad (2)$$

where $x_i, y_i, z_i \in \{0, 1\}$ denotes the i^{th} least significant bits. The Z-order code is obtained by interleaving the bits of the three coordinates. Formally:

$$\mathbf{Z}(x, y, z) = \sum_{i=0}^{d-1} \left(x_i \cdot 2^{3i} + y_i \cdot 2^{3i+1} + z_i \cdot 2^{3i+2} \right) \quad (3)$$

This mapping is efficient for spatial applications as it maintains locality while providing a compact low-dimensional representation of multi-dimensional points [36, 43].

3.2. Z-Order Splat Transformer

Depth Estimation. We begin by predicting the depth maps via a transformer encoder and a depth head, given the N view input images $\mathbf{I} = \{I\}_{i=1}^N$. The structure of the network follows DINOv2-Small [23], which patchifies $\mathbf{I}$ into tokens $\mathbf{t} \in \mathbb{R}^{N \times l \times d}$, where $l = \frac{H \times W}{p}$ is the token number of one image, $p = 14$ is the patch size, and $d = 384$ is the token dimension. Then a DPT-based depth head [26] is applied to the tokens $\mathbf{t}$ to predict the depth maps $\mathbf{D} \in \mathbb{R}^{N \times H \times W}$. Specifically, these tokens $\mathbf{t}$ will first be reshaped into feature maps and then multi-scale representations $\{F_1, F_2, F_3, F_4\}$ from different backbone layers are extracted, where $F_i \in \mathbb{R}^{N \times d_i \times H/p_i \times W/p_i}$ for $i \in \{1, 2, 3, 4\}$. Then these multi-scale feature maps will be fused using linear projections, upsampling, and transformer-style layers to progressively recover spatial resolution and obtain the geometry feature $F_{\text{geom}} \in \mathbb{R}^{N \times 64 \times H \times W}$. Then convolutional layers are used to predict depth maps $\mathbf{D}$ from F_{geom} .

Gaussian Point Representation. Formally, we denote the Gaussian point representation that will be used for the subsequent ZFormer block as $\mathbf{R} = \{\mathbf{P}, \mathbf{F}, \mathbf{I}\}$, where $\mathbf{P} \in \mathbb{R}^{N \times (HW) \times 3}$ is obtained by projecting the depth $\mathbf{D}$ to point

maps in world space via the camera of each input view. These points will be used to provide initialization for Gaussian centers. $\mathbf{F} \in \mathbb{R}^{N \times 96 \times H \times W}$ is the point-wise feature map, which will be used to predict Gaussian parameters. To obtain $\mathbf{F}$, we first reshape the tokens $\mathbf{t}$ into feature maps and then use two unsampling conv layers to obtain the global feature map $F_{\text{global}} \in \mathbb{R}^{N \times 32 \times H \times W}$. We then fuse F_{global} and F_{geom} via concat and a conv layer to obtain $\mathbf{F}$. We also include the pixel color $\mathbf{I}$, which will be used to provide initialization for Gaussian SH parameter.

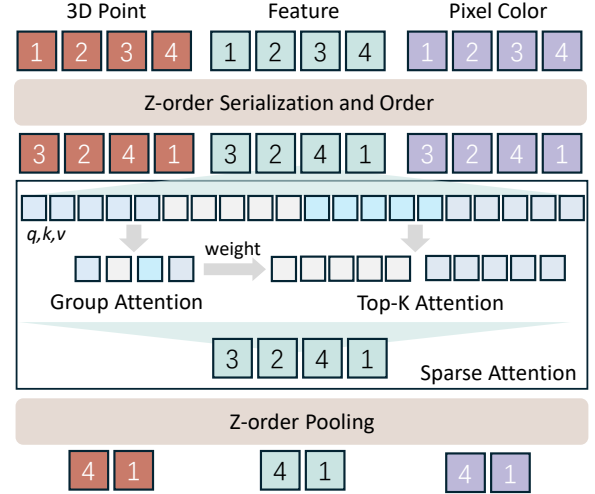


Figure 4. **ZFormer Block.** The ZFormer block begins by serializing and ordering the 3D points, features, and pixel colors using Z-order. The serialized data is then passed through a sparse attention mechanism, including group attention, and top-K attention. After the attention steps, Z-order pooling is applied to further aggregate the features.

ZFormer Blocks. Given the Gaussian point representation $\mathbf{R} = \{\mathbf{P}, \mathbf{F}, \mathbf{I}\}$, initially, the 3D points $\mathbf{P}$ will be merged along the view dimension to obtain $\hat{\mathbf{P}} \in \mathbb{R}^{(NHW) \times 3}$ and then encoded to Z-order code $\mathbf{Z} \in \mathbb{R}^{(NHW) \times 1}$ via Z-order serialization (Eq. (3)). After that, we reorder $\mathbf{R}$ by sorting $\mathbf{Z}$ in ascending order. This procedure arranges the points along the Z-order curve, such that spatially adjacent points in 3D space are also placed in close proximity in the sequence. The resulting ordered set $\mathbf{R} = \{\mathbf{P}_{\text{sorted}}, \mathbf{F}_{\text{sorted}}, \mathbf{I}_{\text{sorted}}\}$ preserves spatial locality and provides a structured input representation.

This structured spatial locality motivates us to design a group attention to aggregate sequential blocks of keys or values into block-level representations to capture local information within each group. Given the input feature sequence $\mathbf{F}_{\text{sorted}}$, we first project it into query, key, and value embeddings:

$$\mathbf{Q} = \mathbf{F}_{\text{sorted}} W_Q, \quad \mathbf{K} = \mathbf{F}_{\text{sorted}} W_K, \quad \mathbf{V} = \mathbf{F}_{\text{sorted}} W_V \quad (4)$$

where $\mathbf{Q}, \mathbf{K}, \mathbf{V} \in \mathbb{R}^{(NHW) \times (96/3)}$. The sequence is partitioned into $B = \frac{N \times H \times W}{L}$ non-overlapping blocks of length L . We then define a block-wise operator $\mathcal{C}$ that performs average pooling over sequential blocks:

$$\mathcal{C}(\mathbf{X})_i = \frac{1}{L} \sum_{j=1}^L \mathbf{X}_{(i-1)L+j} \quad \text{for } i = 1, 2, \dots, B \quad (5)$$

We then apply this operator to all three projected matrices, yielding the block-wise representations:

$$\hat{\mathbf{Q}} = \mathcal{C}(\mathbf{Q}), \quad \hat{\mathbf{K}} = \mathcal{C}(\mathbf{K}), \quad \hat{\mathbf{V}} = \mathcal{C}(\mathbf{V}) \quad (6)$$

The group attention is calculated using the scaled dot-product attention:

$$\text{Attn}_{\text{grp}}(\hat{\mathbf{Q}}, \hat{\mathbf{K}}, \hat{\mathbf{V}}) = \text{softmax} \left(\frac{\hat{\mathbf{Q}}\hat{\mathbf{K}}^\top}{\sqrt{d}} \right) \hat{\mathbf{V}} \quad (7)$$

While this group attention significantly reduces computational complexity and captures spatial locality, the averaging operation may dilute fine-grained information within each block. Our solution is to use the top-k attention strategy [7, 30] to select only the top-k most relevant blocks, while reducing attention complexity, where the key is the selection strategy. Inspired by [42], we use the weight $w = \text{softmax} \left(\frac{\hat{\mathbf{Q}}\hat{\mathbf{K}}^\top}{\sqrt{d}} \right)$ from Eq. (7) to provide differentiable block selection importance scores. Then the top-k attention is formulated as:

$$\text{Attn}_{\text{sel}} = \text{softmax} \left(\frac{\mathbf{Q}\mathbf{K}_{\text{sel}}^\top}{\sqrt{d}} \right) \mathbf{V}_{\text{sel}} \quad (8)$$

Following [42], we then adaptively fuse both attention scores using a gate network g :

$$\mathbf{F}_{\text{gate}} = g_1(\mathbf{F}_{\text{sorted}}) \odot \text{Attn}_{\text{grp}} + g_2(\mathbf{F}_{\text{sorted}}) \odot \text{Attn}_{\text{sel}} \quad (9)$$

Currently, we have the Gaussian point representation $\mathbf{R} = \{\mathbf{P}_{\text{sorted}}, \mathbf{F}_{\text{gate}}, \mathbf{I}_{\text{sorted}}\}$. We then introduce a Z-order pooling to reduce the number of points. For the Z-order code $\mathbf{Z} \in \mathbb{R}^{(NHW) \times 1}$, defining the pooling depth as h , we have $\mathbf{Z} = \mathbf{Z} \gg h$, where $\gg$ denotes the bitwise right shift operator. Points with the same Z-order code are grouped into the same cluster. We then conduct an average pooling in each cluster and add another linear projection for the feature part, and obtain the compressed Gaussian point representation: $\mathbf{R} = \{\mathbf{P}_{\text{pool}}, \mathbf{F}_{\text{pool}}, \mathbf{I}_{\text{pool}}\}$, where $\mathbf{P}_{\text{pool}} \in \mathbb{R}^{M \times 3}$ is obtained by inverting Z-order transformation function Eq. (3), $\mathbf{F}_{\text{pool}} \in \mathbb{R}^{M \times 96}$, and $\mathbf{I}_{\text{pool}} \in \mathbb{R}^{M \times 3}$. Note M is much smaller than (NHW) . This new Gaussian point representation can be further used for the Z-order block to aggregate points and reduce the number of points. Suppose there are two Z-order blocks, we will obtain two Gaussian point representations $\mathbf{R}_{L1}$ and $\mathbf{R}_{L2}$.

Gaussian Head. We use a two-layer MLP network $\mathcal{F}_{\text{head}}$ to predict Gaussian parameters from various layer Gaussian point representations $\mathbf{R}_{L1}$ and $\mathbf{R}_{L2}$:

$$G_{L1}, G_{L2} = \mathcal{F}_{\text{head}}(\mathbf{R}_{L1}), \mathcal{F}_{\text{head}}(\mathbf{R}_{L2}) \quad (10)$$

Two key points have been considered here: first, we use $\mathbf{P}_{\text{pool}}$ as the base Gaussian centers, and the final Gaussian center is $\mu = \mathbf{P}_{\text{pool}} + \Delta\mu$, where $\Delta\mu$ is predicted by $\mathcal{F}_{\text{head}}$. Second, we initialize the SH parameter c using the pixel color $\mathbf{I}_{\text{pool}}$ by converting it into SH coefficients.

3.3. Model Training

We use the pre-trained depth-anything-v2-small [38] $\hat{\mathcal{F}}_{\text{depth}}$ to initialize our depth estimator $\mathcal{F}_{\text{depth}}$. During training, we find that multi-task training with depth estimation favors local details. Thus, rather than fixing the depth estimator, we use $\hat{\mathcal{F}}_{\text{depth}}$ to distill by adding a loss:

$$\mathcal{L}_{\text{depth}} = \left| \mathcal{F}_{\text{depth}}(\mathbf{I}) - \hat{\mathcal{F}}_{\text{depth}}(\mathbf{I}) \right| \quad (11)$$

At the same time, we train the full model with a combination of mean squared error (MSE) and LPIPS [45] losses between the rendered and ground-truth image colors:

$$\mathcal{L}_{\text{color}} = \sum_{i=1}^M [\text{MSE}(\mathcal{R}(G_{Li}, \mathbf{c}), \mathbf{I}_{\text{gt}}) + \text{LPIPS}(\mathcal{R}(G_{Li}, \mathbf{c}), \mathbf{I}_{\text{gt}})] \quad (12)$$

where M is the Z-order block number, $\mathbf{c}$ is the rendered camera, and $\mathcal{R}$ is the Gaussian render.

3.4. Inference with Dense Views

Dense views may be available during inference; however, an excessive number of views can negatively impact inference efficiency. To address this, we propose a Z-order based Maximum Coverage Viewpoint Selection algorithm that effectively reduces redundancy in these dense views, improving efficiency while maintaining performance.

The key of our algorithm is to maximize coverage during view selection. Given a set of point maps $\mathcal{V}$ from N input views, the algorithm first applies Z-order serialization to encode each point map into a compact point-based representation. The Z-order coverage for each viewpoint is then stored in a max-heap. In each iteration, the algorithm selects the viewpoint offering the largest coverage from the heap and adds it to the set of selected viewpoints $\mathcal{S}$, while updating the covered set $\mathcal{C}$. If no additional viewpoints in the heap contribute sufficient new coverage, the algorithm terminates. By adopting this greedy strategy, the algorithm incrementally selects the most informative viewpoints, thereby reducing redundancy and enhancing computational efficiency. We provide a detailed description of the algorithmic process in Supplementary Algorithm 1.

Method	2 Views			4 Views			8 Views			12 Views		
	PSNR↑	SSIM↑	LPIPS↓	PSNR↑	SSIM↑	LPIPS↓	PSNR↑	SSIM↑	LPIPS↓	PSNR↑	SSIM↑	LPIPS↓
RealEstate10K, 360×640												
3DGS [9]	16.80	0.591	0.351	21.29	0.753	0.279	23.02	0.783	0.257	26.73	0.871	0.144
MipSplatting [41]	18.09	0.703	0.310	23.46	0.801	0.214	24.87	0.812	0.198	27.34	0.891	0.117
DepthSplat [37]	26.03	0.873	0.158	26.13	0.872	0.157	26.17	0.876	0.152	26.33	0.880	0.143
AnySplat [8]	22.55	0.757	0.229	25.86	0.824	0.152	26.71	0.886	0.131	26.94	0.892	0.122
Ours#L1	26.43	0.873	0.147	26.68	0.891	0.142	27.25	0.897	0.123	28.56	0.901	0.110
Ours#L2	26.42	0.870	0.159	26.59	0.884	0.144	26.78	0.884	0.132	28.12	0.898	0.120
DL3DV 256×448												
3DGS [9]	18.79	0.571	0.311	19.51	0.656	0.288	21.70	0.736	0.200	23.63	0.810	0.159
MipSplatting [41]	19.10	0.657	0.288	20.28	0.649	0.246	22.47	0.713	0.235	23.79	0.819	0.137
DepthSplat [37]	20.23	0.679	0.268	23.08	0.760	0.183	24.15	0.810	0.141	25.74	0.844	0.131
AnySplat [8]	20.65	0.669	0.283	22.85	0.746	0.234	24.38	0.823	0.167	24.90	0.856	0.136
Ours#L1	23.06	0.774	0.201	24.95	0.845	0.143	26.22	0.855	0.137	27.09	0.892	0.124
Ours#L2	22.99	0.770	0.206	24.55	0.830	0.153	25.53	0.846	0.142	26.62	0.876	0.131

Table 1. **Quantitative Comparison with Varying Numbers of Views.** We highlight the best results in red and the second-best in yellow.

4. Experiment

Datasets. We evaluate our method using three large-scale datasets: the 360×640 resolution RealEstate10K [49], the 256×448 resolution DL3DV [16], and the 256×256 resolution ACID [17]. The training and testing splits follow the protocol of DepthSplat [37]. For RealEstate10K and DL3DV, we use the provided estimated camera intrinsic and extrinsic parameters, while for ACID, we derive the camera information using VGGT [31]. The main experiments are conducted on the RealEstate10K and DL3DV datasets. ACID is not used for training; instead, it is employed exclusively for cross-dataset evaluation.

Implementation Details. In our framework, we utilize two ZFormer blocks. The block size for group attention is set to 32, while for top-k attention, we select half of the total number of blocks. To accelerate the attention computation, we use FlashAttention [28]. The Z-order pooling depth is set to 2. Our method is implemented in PyTorch, and we optimize the model using the AdamW [20] optimizer with a cosine learning rate schedule. We use a lower learning rate of 2×10^{-6} for the depth estimation branch, while the ZFormer blocks and the Gaussian head are trained with a learning rate of 2×10^{-4} . The model is trained on $8 \times$ A100 GPUs for 100K iterations with a batch size of 2, taking approximately 2 days. For quantitative results, we report standard image quality metrics, including pixel-level PSNR, patch-level SSIM [35], and feature-level LPIPS [45].

4.1. Comparisons

We compare our method with two classical optimization-based methods, 3DGS [9] and MipSplatting [41], as well as two representative feed-forward methods, DepthSplat [37] and AnySplat [8]. For all methods, we use the same input views with an identical resolution for comparison. Additionally, for the feed-forward methods, we use the same

training data and the official training script during the training process.

Comparisons on Fixed View Input. We compare the results for different numbers of input views (2, 4, 8, and 12) in Tab. 1. Our method with a layer-one Z-order block (Ours#L1) consistently outperforms related works across all metrics and for all view counts. In most cases, our method with two Z-order block compressions (Ours#L2) outperforms the comparison methods, achieving better results across various metrics and input configurations. Our improvements are particularly significant with fewer input views, especially when using only two views. This demonstrates that our model is more robust to sparse views. We also provide a visual comparison in Fig. 5, where our method demonstrates better performance, surpassing the baselines in capturing sharp edges and intricate details.

Comparisons on Variable Input. In Tab. 2, we evaluate the performance of various feed-forward methods across different view inputs, ranging from 2 to 12 views. All methods are initially trained on the RealEstate10K dataset for 100K iterations and then fine-tuned on the DL3DV dataset for an additional 50K iterations, with view inputs varying from 2 to 12. Both Ours#L1 and Ours#L2 outperform the baseline methods, DepthSplat and AnySplat, across all three metrics. Ours#L1 achieves the best overall performance, while Ours#L2 slightly trails behind, demonstrating the superior image quality, structural consistency, and perceptual fidelity of our approach.

Cross-Dataset Comparisons. We assess the cross-dataset generalization capability by testing the RealEstate10K and DL3DV pre-trained models on the ACID dataset in Tab. 3. Ours#L1 achieves the best performance in both RealEstate10K→ACID and DL3DV→ACID evaluations. Ours#L2 also shows strong results, closely trailing Ours#L1 across all metrics. These findings highlight the superior

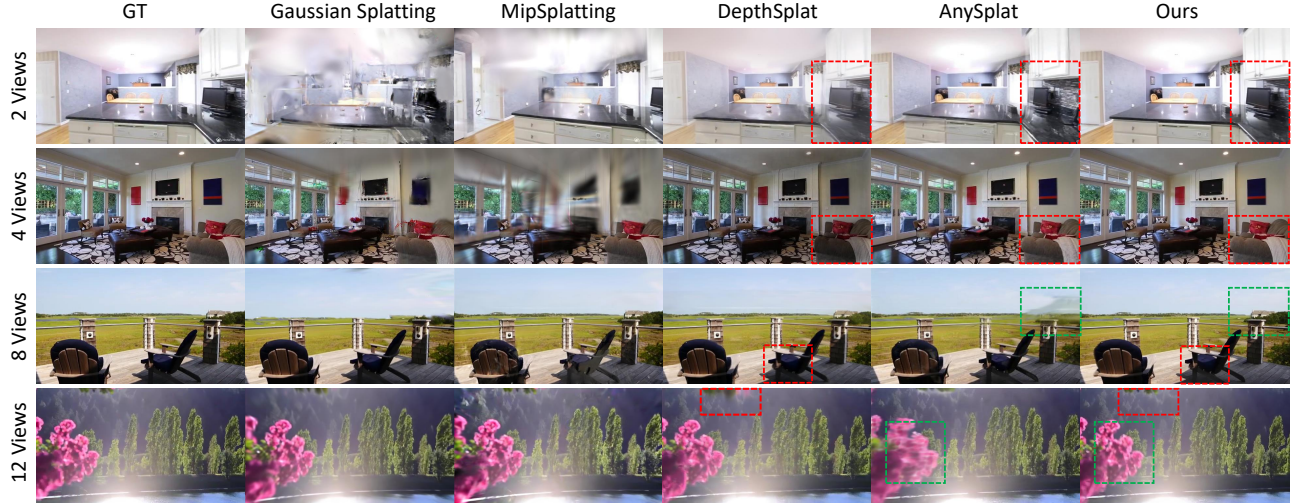


Figure 5. **Visual Comparison.** Our method achieves better performance in capturing sharp edges and intricate details.

generalization ability of our methods across datasets.

Inference Time and Number of Gaussians Comparisons.

In Tab. 4, our methods, Ours#L1 and Ours#L2, significantly outperform the baseline methods in both inference time and the number of Gaussian primitives. Notably, our method is approximately 1,000 times faster than 3D-GS and MipSplatting, with a $3\times$ reduction in Gaussian primitives for 2 views. Additionally, our approach achieves a 2- to 3-fold reduction in Gaussian primitives compared to DepthSplat and AnySplat.

4.2. Ablation Study

We conduct six ablation experiments using 12 input views in Tab. 5, Tab. 6, Fig. 6, and Fig. 7 to evaluate the impact of various design choices on the performance of our model, including model structure, training strategy, Z-order layer selection, and view selection during inference.

Model Design. In Tab. 5, replacing the ZFormer blocks with convolutional layers (Ours w/o Z-order) leads to a noticeable decrease in all metrics (resulting in the worst performance), suggesting that the ZFormer block is essential in our framework. Using Z-order while replacing sparse attention with a full attention layer (Ours w/o SA) also results in a clear performance drop compared to our full model, indicating that sparse attention plays a crucial role in our model design. These observations can also be confirmed in Fig. 6, where the results tend to be coarse both globally and locally without ZFormer or sparse attention.

Training Strategy. During training, we found it important to initialize SH parameters using the pixel color from the input views, as this enhances training stability and leads to better convergence (refer to Ours w/o SH in Tab. 5 and Fig. 6). Moreover, by comparing our method without multi-task depth training (Ours-Fix-Depth) to the full model in

Method	2-12 Views		
	PSNR $\uparrow$	SSIM $\uparrow$	LPIPS $\downarrow$
DepthSplat [37]	26.11	0.871	0.151
AnySplat [8]	26.54	0.875	0.133
Ours#L1	28.07	0.890	0.125
Ours#L2	27.62	0.881	0.129

Table 2. **Comparison on Variable Input.** We train and evaluate the performance across various view inputs (2 to 12 views).

Tab. 5 and Fig. 6, we find that it is important not to fix depth estimation during training. This is because 2D image-based depth estimation is not always accurate, and joint training allows the model to compensate for such inaccuracies.

Z-order Layer Selection. We explain the selection of two Z-order layers in Fig. 7. We select two Z-order layers in our framework because they strike a balance between preventing degradation and maintaining a low number of GS primitives (also see Tab. 4). These two layers help preserve fine details and overall structure, as evidenced by the degradation observed when more than two layers are used. The optimal performance achieved with two layers reflects a careful trade-off between quality and efficiency.

Inference View Selection. In Tab. 6, we investigate the impact of different view selection strategies on the performance and efficiency of our model. Our Z-order selection consistently outperforms random selection in terms of image quality. It also provides a more efficient alternative to using all views, with slightly longer inference times compared to random selection. This suggests our Z-order selection is an effective method for balancing both performance and computational efficiency.



Figure 6. **Visual Ablation Study.** Results of ablations on different components of our method.

Method	RealEstate10K $\rightarrow$ ACID			DL3DV $\rightarrow$ ACID		
	PSNR $\uparrow$	SSIM $\uparrow$	LPIPS $\downarrow$	PSNR $\uparrow$	SSIM $\uparrow$	LPIPS $\downarrow$
DepthSplat [37]	26.05	0.810	0.181	25.58	0.796	0.203
AnySplat [8]	22.71	0.685	0.298	23.64	0.737	0.242
Ours#L1	27.56	0.853	0.172	27.34	0.845	0.169
Ours#L2	27.01	0.824	0.183	26.95	0.831	0.187

Table 3. **Cross-Dataset Evaluation.** We assess the cross-dataset generalization capability by testing the RealEstate10K and DL3DV pre-trained models on the ACID dataset.

Method	2 Views		12 Views	
	Opt./Inf.	#GS	Opt./Inf.	#GS
3DGS [9]	2m15s	6.27	8m21s	8.51
MipSplatting [41]	1m18s	4.39	5m14s	8.14
DepthSplat [37]	0.142s	4.61	0.384s	27.6
AnySplat [8]	0.692s	3.53	1.212s	13.2
Ours#L1	0.123s	2.85	0.337s	17.8
Ours#L2	0.135s	1.42	0.355s	8.05

Table 4. **Comparison of Runtime and the Number of Gaussian Primitives.** Values are reported in milliseconds (ms) for 2 and 12 views, with a resolution of 360×640 . #GS denotes the number of Gaussian primitives ($\times 10^5$).

Method	PSNR $\uparrow$	SSIM $\uparrow$	LPIPS $\downarrow$
Ours w/o Z-order	24.86	0.794	0.225
Ours w/o SA	26.79	0.847	0.174
Ours w/o SH	27.81	0.874	0.129
Ours-Fix-Depth	28.15	0.893	0.119
Ours	28.56	0.901	0.110

Table 5. **Ablation Study.** We have conducted ablations on different components of our method.

5. Conclusion

In this work, we have proposed a Z-order transformer framework for efficient 3DGS and novel view synthesis. By employing a Z-order strategy, we arrange Gaussian primitives into a spatial sequence, enabling effective context modeling and efficient prediction in a single forward pass. Furthermore, we propose a sparse attention mechanism to reduce the complexity of processing large-scale Gaussian data, significantly enhancing both the efficiency and quality of the reconstruction. Through extensive experiments, we have demonstrated that our method outperforms existing

Inp. Views	Selection	PSNR $\uparrow$	SSIM $\uparrow$	LPIPS $\downarrow$	Time(s) $\downarrow$
24	NA.	28.91	0.906	0.102	0.622
	RS. 16	27.97	0.891	0.113	0.417
	ZS. 16	28.73	0.903	0.108	0.448
16	NA.	28.67	0.901	0.110	0.417
	RS. 8	27.06	0.876	0.125	0.255
	ZS. 8	28.07	0.898	0.116	0.272

Table 6. **Ablation Study with Different Selection Strategies.** NA. indicates that all views will be used during inference, RS. refers to random selection, while ZS. denotes our Z-order-based view selection method. The input resolution is 360×640 .



Figure 7. **Ablation Study on Layer Selection.** We use two Z-order blocks in our framework to prevent degradation while achieving a lower number of GS primitives.

state-of-the-art approaches. Notably, our method achieves faster inference times while requiring fewer Gaussian primitives for high-quality novel view synthesis. In summary, our Z-order transformer framework advances 3DGS, offering an efficient solution for real-time, high-quality novel view synthesis with fewer Gaussian primitives.

References

- [1] David Charatan, Sizhe Lester Li, Andrea Tagliasacchi, and Vincent Sitzmann. pixelsplat: 3d gaussian splats from image pairs for scalable generalizable 3d reconstruction. In *Proceedings of the IEEE/CVF conference on computer vision and pattern recognition*, pages 19457–19467, 2024. 1, 2, 11
- [2] Yuedong Chen, Haofei Xu, Chuanxia Zheng, Bohan Zhuang, Marc Pollefeys, Andreas Geiger, Tat-Jen Cham, and Jianfei Cai. Mvsplat: Efficient 3d gaussian splatting from sparse multi-view images. In *European Conference on Computer Vision*, pages 370–386. Springer, 2024. 1, 2, 11
- [3] Kai Cheng, Xiaoxiao Long, Kaizhi Yang, Yao Yao, Wei Yin, Yuexin Ma, Wenping Wang, and Xuejin Chen. Gaussianpro: 3d gaussian splatting with progressive propagation. In *Forty-first International Conference on Machine Learning*, 2024. 2
- [4] Michael Connor and Piyush Kumar. Fast construction of k-nearest neighbor graphs for point clouds. *IEEE transactions on visualization and computer graphics*, 16(4):599–608, 2010. 3
- [5] Shuangkang Fang, I Shen, Takeo Igarashi, Yufeng Wang, ZeSheng Wang, Yi Yang, Wenrui Ding, Shuchang Zhou, et al. Nerf is a valuable assistant for 3d gaussian splatting. In *Proceedings of the IEEE/CVF International Conference on Computer Vision*, pages 26230–26240, 2025. 13
- [6] Guofeng Feng, Siyan Chen, Rong Fu, Zimu Liao, Yi Wang, Tao Liu, Boni Hu, Linning Xu, Zhilin Pei, Hengjie Li, et al. Flashgs: Efficient 3d gaussian splatting for large-scale and high-resolution rendering. In *Proceedings of the Computer Vision and Pattern Recognition Conference*, pages 26652–26662, 2025. 2
- [7] Ankit Gupta, Guy Dar, Shaya Goodman, David Ciprut, and Jonathan Berant. Memory-efficient transformers via top-k attention. In *Proceedings of the Second Workshop on Simple and Efficient Natural Language Processing*, pages 39–52, 2021. 5
- [8] Lihan Jiang, Yucheng Mao, Linning Xu, Tao Lu, Kerui Ren, Yichen Jin, Xudong Xu, Mulin Yu, Jiangmiao Pang, Feng Zhao, et al. Anysplat: Feed-forward 3d gaussian splatting from unconstrained views. *arXiv preprint arXiv:2505.23716*, 2025. 1, 2, 3, 6, 7, 8, 11
- [9] Bernhard Kerbl, Georgios Kopanas, Thomas Leimkühler, and George Drettakis. 3d gaussian splatting for real-time radiance field rendering. *ACM Trans. Graph.*, 42(4):139–1, 2023. 1, 2, 6, 8
- [10] Hanyang Kong, Xingyi Yang, and Xinchao Wang. Rogsplat: Robust gaussian splatting via generative priors. In *Proceedings of the IEEE/CVF International Conference on Computer Vision*, pages 25735–25745, 2025. 2
- [11] Ken CK Lee, Wang-Chien Lee, Baihua Zheng, Huajing Li, and Yuan Tian. Z-sky: an efficient skyline query processing framework based on z-order. *The VLDB Journal*, 19(3):333–362, 2010. 3
- [12] Haolin Li, Jinyang Liu, Mario Sznaiier, and Octavia Camps. 3d-hgs: 3d half-gaussian splatting. In *Proceedings of the Computer Vision and Pattern Recognition Conference*, pages 10996–11005, 2025. 2
- [13] Dingkan Liang, Xin Zhou, Wei Xu, Xingkui Zhu, Zhikang Zou, Xiaoqing Ye, Xiao Tan, and Xiang Bai. Pointmamba: A simple state space model for point cloud analysis. *Advances in neural information processing systems*, 37:32653–32677, 2024. 3
- [14] Jingyu Lin, Jiaqi Gu, Lubin Fan, Bojian Wu, Yujing Lou, Renjie Chen, Ligang Liu, and Jieping Ye. Hybrids: Decoupling transients and statics with 2d and 3d gaussian splatting. In *Proceedings of the Computer Vision and Pattern Recognition Conference*, pages 788–797, 2025. 13
- [15] Peter Lindstrom and Valerio Pascucci. Visualization of large terrains made easy. In *Proceedings Visualization, 2001. VIS'01.*, pages 363–374. IEEE, 2001. 3
- [16] Lu Ling, Yichen Sheng, Zhi Tu, Wentian Zhao, Cheng Xin, Kun Wan, Lantao Yu, Qianyu Guo, Zixun Yu, Yawen Lu, et al. D3dv-10k: A large-scale scene dataset for deep learning-based 3d vision. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*, pages 22160–22169, 2024. 6
- [17] Andrew Liu, Richard Tucker, Varun Jampani, Ameesh Makadia, Noah Snavely, and Angjoo Kanazawa. Infinite nature: Perpetual view generation of natural scenes from a single image. In *Proceedings of the IEEE/CVF International Conference on Computer Vision*, pages 14458–14467, 2021. 6
- [18] Lei Liu, Can Wang, Zhenghao Chen, Zhenhua Liu, and Dong Xu. 4dgs-craft: Consistent and interactive 4d gaussian splatting editing. *IEEE Transactions on Circuits and Systems for Video Technology*, 2026. 1
- [19] Yifan Liu, Keyu Fan, Weihao Yu, Chenxin Li, Hao Lu, and Yixuan Yuan. Monosplat: Generalizable 3d gaussian splatting from monocular depth foundation models. In *Proceedings of the Computer Vision and Pattern Recognition Conference*, pages 21570–21579, 2025. 1, 2, 11
- [20] Ilya Loshchilov and Frank Hutter. Decoupled weight decay regularization. In *International Conference on Learning Representations*, 2017. 6
- [21] Guy M Morton. *A computer oriented geodetic data base and a new technique in file sequencing*. International Business Machines Company, 1966. 2, 3, 4
- [22] Thomas Müller, Alex Evans, Christoph Schied, and Alexander Keller. Instant neural graphics primitives with a multiresolution hash encoding. *ACM transactions on graphics (TOG)*, 41(4):1–15, 2022. 13
- [23] Maxime Oquab, Timothée Darcet, Théo Moutakanni, Huy V Vo, Marc Szafraniec, Vasil Khalidov, Pierre Fernandez, Daniel HAZIZA, Francisco Massa, Alaaeldin El-Nouby, et al. Dinov2: Learning robust visual features without supervision. *Transactions on Machine Learning Research*, 2025. 4, 12
- [24] Jack A Orenstein. Spatial query processing in an object-oriented database system. In *Proceedings of the 1986 ACM SIGMOD international conference on Management of data*, pages 326–336, 1986. 3
- [25] Jack A Orenstein. Redundancy in spatial databases. *ACM SIGMOD Record*, 18(2):295–305, 1989. 3
- [26] René Ranftl, Alexey Bochkovskiy, and Vladlen Koltun. Vision transformers for dense prediction. In *Proceedings of*

- the *IEEE/CVF international conference on computer vision*, pages 12179–12188, 2021. 4, 12
- [27] Xuanchi Ren, Yifan Lu, Hanxue Liang, Zhangjie Wu, Huan Ling, Mike Chen, Sanja Fidler, Francis Williams, and Jiahui Huang. Scube: Instant large-scale scene reconstruction using voxplats. *Advances in Neural Information Processing Systems*, 37:97670–97698, 2024. 2, 3
- [28] Jay Shah, Ganesh Bikshandi, Ying Zhang, Vijay Thakkar, Pradeep Ramani, and Tri Dao. Flashattention-3: Fast and accurate attention with asynchrony and low-precision. *Advances in Neural Information Processing Systems*, 37:68658–68685, 2024. 6
- [29] Marco Tarini, Paolo Cignoni, and Claudio Montani. Ambient occlusion and edge cueing for enhancing real time molecular visualization. *IEEE transactions on visualization and computer graphics*, 12(5):1237–1244, 2006. 3
- [30] Apoorv Vyas, Angelos Katharopoulos, and François Fleuret. Fast transformers with clustered attention. *Advances in Neural Information Processing Systems*, 33:21665–21674, 2020. 5
- [31] Jianyuan Wang, Minghao Chen, Nikita Karaev, Andrea Vedaldi, Christian Rupprecht, and David Novotny. Vggt: Visual geometry grounded transformer. In *Proceedings of the Computer Vision and Pattern Recognition Conference*, pages 5294–5306, 2025. 6, 12
- [32] Peng-Shuai Wang. Octformer: Octree-based transformers for 3d point clouds. *ACM Transactions on Graphics (TOG)*, 42(4):1–11, 2023. 3
- [33] Yiming Wang, Lucy Chai, Xuan Luo, Michael Niemeyer, Manuel Lagunas, Stephen Lombardi, Siyu Tang, and Tiancheng Sun. Learning efficient fuse-and-refine for feed-forward 3d gaussian splatting. In *The Thirty-ninth Annual Conference on Neural Information Processing Systems*, 2025. 2, 3
- [34] Zipeng Wang and Dan Xu. Hyrf: Hybrid radiance fields for memory-efficient and high-quality novel view synthesis. *arXiv preprint arXiv:2509.17083*, 2025. 13
- [35] Zhou Wang, Alan C Bovik, Hamid R Sheikh, and Eero P Simoncelli. Image quality assessment: from error visibility to structural similarity. *IEEE transactions on image processing*, 13(4):600–612, 2004. 6
- [36] Xiaoyang Wu, Li Jiang, Peng-Shuai Wang, Zhijian Liu, Xihui Liu, Yu Qiao, Wanli Ouyang, Tong He, and Hengshuang Zhao. Point transformer v3: Simpler faster stronger. In *Proceedings of the IEEE/CVF conference on computer vision and pattern recognition*, pages 4840–4851, 2024. 3, 4
- [37] Hao-fei Xu, Songyou Peng, Fang-jinhua Wang, Hermann Blum, Daniel Barath, Andreas Geiger, and Marc Pollefeys. Depthplat: Connecting gaussian splatting and depth. In *Proceedings of the Computer Vision and Pattern Recognition Conference*, pages 16453–16463, 2025. 1, 2, 6, 7, 8, 11
- [38] Lihe Yang, Bingyi Kang, Zilong Huang, Zhen Zhao, Xiaogang Xu, Jiashi Feng, and Hengshuang Zhao. Depth anything v2. *Advances in Neural Information Processing Systems*, 37:21875–21911, 2024. 5, 12
- [39] Botao Ye, Sifei Liu, Hao-fei Xu, Xueting Li, Marc Pollefeys, Ming-Hsuan Yang, and Songyou Peng. No pose, no problem: Surprisingly simple 3d gaussian splats from sparse unposed images. In *The Thirteenth International Conference on Learning Representations*, 2025. 1, 2, 11
- [40] Vickie Ye, Ruilong Li, Justin Kerr, Matias Turkulainen, Brent Yi, Zhuoyang Pan, Otto Seiskari, Jianbo Ye, Jeffrey Hu, Matthew Tancik, et al. gsplat: An open-source library for gaussian splatting. *Journal of Machine Learning Research*, 26(34):1–17, 2025. 1, 2
- [41] Zehao Yu, Anpei Chen, Binbin Huang, Torsten Sattler, and Andreas Geiger. Mip-splatting: Alias-free 3d gaussian splatting. In *Proceedings of the IEEE/CVF conference on computer vision and pattern recognition*, pages 19447–19456, 2024. 1, 2, 6, 8
- [42] Jingyang Yuan, Huazuo Gao, Damai Dai, Junyu Luo, Liang Zhao, Zhengyan Zhang, Zhenda Xie, YX Wei, Lean Wang, Zhiping Xiao, et al. Native sparse attention: Hardware-aligned and natively trainable sparse attention. *arXiv preprint arXiv:2502.11089*, 2025. 5
- [43] QIUHAO Zeng, Jerry Huang, Peng Lu, Gezheng Xu, Boxing Chen, Charles Ling, and Boyu Wang. Zeta: Leveraging z-order curves for efficient top-k attention. In *The Thirteenth International Conference on Learning Representations*, 2025. 4
- [44] Dongbin Zhang, Chuming Wang, Weitao Wang, Peihao Li, Minghan Qin, and Haoqian Wang. Gaussian in the wild: 3d gaussian splatting for unconstrained image collections. In *European Conference on Computer Vision*, pages 341–359. Springer, 2024. 2
- [45] Richard Zhang, Phillip Isola, Alexei A Efros, Eli Shechtman, and Oliver Wang. The unreasonable effectiveness of deep features as a perceptual metric. In *Proceedings of the IEEE conference on computer vision and pattern recognition*, pages 586–595, 2018. 5, 6
- [46] Shangzhan Zhang, Jianyuan Wang, Yinghao Xu, Nan Xue, Christian Rupprecht, Xiaowei Zhou, Yujun Shen, and Gordon Wetzstein. Flare: Feed-forward geometry, appearance and camera estimation from uncalibrated sparse views. In *Proceedings of the Computer Vision and Pattern Recognition Conference*, pages 21936–21947, 2025. 1, 2, 11
- [47] Shunyu Zhang, Boyao Zhou, Ruizhi Shao, Boning Liu, Shengping Zhang, Liqiang Nie, and Yebin Liu. Gps-gaussian: Generalizable pixel-wise 3d gaussian splatting for real-time human novel view synthesis. In *Proceedings of the IEEE/CVF conference on computer vision and pattern recognition*, pages 19680–19690, 2024. 1, 2
- [48] Hongliang Zhong, Can Wang, Jingbo Zhang, and Jing Liao. Generative object insertion in gaussian splatting with a multi-view diffusion model. *Visual Informatics*, 9(2):100238, 2025. 1
- [49] Tinghui Zhou, Richard Tucker, John Flynn, Graham Fyffe, and Noah Snavely. Stereo magnification: learning view synthesis using multiplane images. *ACM Transactions on Graphics (TOG)*, 37(4):1–12, 2018. 6, 11

Supplemental Material

1. Z-order Based Maximum Coverage Viewpoint Selection Algorithm

In the main paper, we provide a high-level introduction to the proposed Z-order based Maximum Coverage Viewpoint Selection algorithm. Here, we include the detailed description of this algorithm, as shown in Algorithm 1. The proposed algorithm aims to select at most M viewpoints from a dense candidate set $\mathcal{V} = \{V_1, \dots, V_N\}$ such that their union maximally covers the underlying 3D scene while avoiding redundant views. For each candidate viewpoint V_i , we first obtain its associated point cloud P_i and discretize the 3D space into a regular grid with a cell size δ . Each point $p \in P_i$ is mapped to its grid cell and then serialized into a one-dimensional key using a Z-order encoding $\mathcal{Z}(\phi(p/\delta))$, which preserves spatial locality, where ϕ denotes the quantization from continuous 3D coordinates to integer grid indices, implemented as component-wise floor $\phi(p/\delta) = \lfloor p/\delta \rfloor$. The resulting set of encoded grid cells for viewpoint V_i is denoted as S_i , and its cardinality $|S_i|$ reflects the raw coverage of that viewpoint. We then initialize a max-heap $\mathcal{H}$ with tuples $(|S_i|, i)$ for all viewpoints, so that the viewpoint with the largest coverage can be efficiently retrieved. The algorithm proceeds in a greedy fashion for at most M iterations. At each iteration, we repeatedly pop the current best candidate (c, j) from $\mathcal{H}$ and compute its marginal contribution $\Delta = S_j \setminus \mathcal{C}$ with respect to the already covered set $\mathcal{C}$. If $|\Delta| > 0$, viewpoint V_j is accepted: we update the global coverage $\mathcal{C} \leftarrow \mathcal{C} \cup \Delta$ and append j to the selected viewpoint index set $\mathcal{S}$. If the heap is exhausted without finding a viewpoint that adds new coverage, the selection process terminates early, indicating that remaining viewpoints are completely redundant. After each successful selection, we rebuild the heap by recomputing, for every remaining candidate viewpoint V_j , its updated marginal coverage $\Delta_j = S_j \setminus \mathcal{C}$ and pushing only those with $|\Delta_j| > 0$ back into a new heap $\mathcal{H}_{\text{new}}$. Overall, this Z-order based greedy scheme incrementally selects the most informative viewpoints under a discretized spatial coverage criterion, effectively eliminating redundant views and improving inference efficiency while avoiding significant performance degradation.

In Tab. 6 of the main paper, we investigate the impact of different view selection strategies. Here, we further include a denser input setting to extend Tab. 6 and evaluate performance, as shown in Tab. S7-top. For the denser 64-view input setting, our method still performs well, exhibiting only a minor performance drop while removing redundant views. The efficiency improvement is also more pronounced in this case, as we discard half of the input views.

Inp. Views	Selection	PSNR $\uparrow$	SSIM $\uparrow$	LPIPS $\downarrow$	Time(s) $\downarrow$
64	NA.	29.44	0.911	0.098	1.891
	RS. 16	28.50	0.897	0.118	0.421
	ZS. 16	29.13	0.908	0.106	0.498
24	NA.	28.91	0.906	0.102	0.622
	RS. 16	27.97	0.891	0.113	0.417
	ZS. 16	28.73	0.903	0.108	0.448
16	NA.	28.67	0.901	0.110	0.417
	RS. 8	27.06	0.876	0.125	0.255
	ZS. 8	28.07	0.898	0.116	0.272

Table S7. **Ablation Study with Different Selection Strategies.** NA. indicates that all views will be used during inference, RS. refers to random selection, while ZS. denotes our Z-order-based view selection method. The input resolution is 360×640 .

Method	PSNR $\uparrow$	SSIM $\uparrow$	LPIPS $\downarrow$
NoPoSplat [39]	27.41	0.884	0.116
FLARE [46]	23.78	0.801	0.191
MonoSplat [19]	26.68	0.875	0.123
MVSplat [2]	26.39	0.869	0.128
pixelSplat [1]	25.89	0.858	0.142
Ours	27.89	0.892	0.110

Table S8. **More Comparisons.** We present additional comparison results on the RealEstate10K dataset at a resolution of 256×256 with 2 input views.

2. More Comparisons with Related Works

While our comparison baseline, AnySplat [8], has already been shown to outperform NoPoSplat [39] and FLARE [46], and DepthSplat [37] has been shown to surpass MVSplat [2] and PixelSplat [1], we have also included direct comparisons with these methods, as well as MonoSplat [19], to further validate the effectiveness of our approach. All methods are trained and evaluated on the RealEstate10K dataset [49] using the same training and testing split at a resolution of 256×256 with 2 input views. As summarized in Table S8, our method consistently delivers better reconstruction quality (higher PSNR and SSIM, lower LPIPS), demonstrating that our design provides clear advantages over these related approaches.

3. Ablation Studies of Sparse Attention

To quantify the contribution of each component in our sparse attention module, we perform ablation experiments on the RealEstate10K dataset with 12 input views under

Algorithm 1 Z-order Based Maximum Coverage Viewpoint Selection with Max-Heap

Require: Viewpoint set $\mathcal{V} = \{V_1, V_2, \dots, V_N\}$, Number of viewpoints to select M , Grid size δ

Ensure: Selected viewpoint indices $\mathcal{S}$, final coverage $\mathcal{C}$

```

1:  $\mathcal{C} \leftarrow \emptyset$   $\triangleright$  Set of covered grid cells
2:  $\mathcal{S} \leftarrow \emptyset$   $\triangleright$  Set of selected viewpoints
3:  $\mathcal{H} \leftarrow \emptyset$   $\triangleright$  Max-heap (coverage, viewpoint index)
4: for  $i = 1$  to  $N$  do
5:    $P_i \leftarrow$  Get point cloud for viewpoint  $V_i$ 
6:    $S_i \leftarrow \{\mathcal{Z}(\phi(p/\delta)) \mid p \in P_i\}$   $\triangleright$  Z-order
     serialization with grid size  $\delta$ 
7:   push  $(|S_i|, i)$  into  $\mathcal{H}$   $\triangleright$  Push initial coverage into
     heap
8: end for
9: for  $k = 1$  to  $M$  do
10:  if  $\mathcal{H} = \emptyset$  then
11:    break
12:  end if
13:  found  $\leftarrow$  False
14:  while  $\mathcal{H} \neq \emptyset$  and not found do
15:     $(c, j) \leftarrow$  pop from  $\mathcal{H}$   $\triangleright$  Get max coverage
    candidate
16:     $\Delta \leftarrow S_j \setminus \mathcal{C}$   $\triangleright$  Find new covered grid cells for
    viewpoint  $j$ 
17:    if  $|\Delta| > 0$  then
18:       $\mathcal{C} \leftarrow \mathcal{C} \cup \Delta$   $\triangleright$  Add new covered grid cells
19:       $\mathcal{S} \leftarrow \mathcal{S} \cup \{j\}$   $\triangleright$  Add viewpoint  $j$  to
    selected viewpoints
20:      found  $\leftarrow$  True
21:      break
22:    end if
23:  end while
24:  if not found then
25:    break  $\triangleright$  No more coverage can be added
26:  end if
27:   $\mathcal{H}_{\text{new}} \leftarrow \emptyset$ 
28:  while  $\mathcal{H} \neq \emptyset$  do
29:     $(c, j) \leftarrow$  pop from  $\mathcal{H}$ 
30:     $\Delta_j \leftarrow S_j \setminus \mathcal{C}$   $\triangleright$  Recalculate new coverage for
    viewpoint  $j$ 
31:    if  $|\Delta_j| > 0$  then
32:      push  $(|\Delta_j|, j)$  into  $\mathcal{H}_{\text{new}}$ 
33:    end if
34:  end while
35:   $\mathcal{H} \leftarrow \mathcal{H}_{\text{new}}$ 
36: end for
37: return  $\mathcal{S}, |\mathcal{C}|$ 

```

the same training and evaluation settings as in the main paper. As reported in Table S9, disabling the selection mechanism (“Ours w/o sel”) degrades performance, demonstrat-

Method	PSNR $\uparrow$	SSIM $\uparrow$	LPIPS $\downarrow$
Ours w/o sel	26.79	0.847	0.174
Ours w/o SA	26.79	0.847	0.174
Ours	28.56	0.901	0.110

Table S9. **Ablation Studies of Sparse Attention.** We conduct ablation experiments on different components of our sparse attention module. Experiments are conducted on the RealEstate10K dataset at a resolution of 360×640 with 12 input views. Note that we do not perform an ablation of the group attention removal, as the selection attention depends on it.

Method	PSNR $\uparrow$	SSIM $\uparrow$	LPIPS $\downarrow$	Time(s) $\downarrow$
with VGGT [31]	28.81	0.907	0.108	0.815
Ours	28.56	0.901	0.110	0.337

Table S10. **Comparisons with and without the VGGT Backbone.** Using a pre-trained VGGT as the feature and depth extractor slightly improves performance but requires more inference time. Experiments are conducted on the RealEstate10K dataset at a resolution of 360×640 with 12 input views.

ing that adaptively selecting informative tokens is important for effective sparse attention. When we remove the entire sparse attention module (“Ours w/o SA”), the performance degrades the most, confirming that group attention and selective attention jointly contribute to the overall gain. The complete model achieves the best reconstruction quality, with the highest PSNR/SSIM and the lowest LPIPS, demonstrating the effectiveness of our sparse attention design. Note that we do not perform an ablation of the group attention removal, as the selection attention depends on it.

4. Analysis of Using the VGGT Backbone

We also considered using VGGT [31] as the backbone for feature extraction instead of the depth-anything-v2-small [38]. Since VGGT also utilizes DINOv2 [23] and DPT head [26] structures, we fused the global and geometric features extracted from VGGT. The experiment was also conducted on the RealEstate10K dataset with 12 input views. As shown in Table S10, using a pre-trained VGGT backbone slightly improves reconstruction quality (higher PSNR/SSIM and lower LPIPS), likely due to VGGT being trained on a larger dataset. However, the inference time is significantly longer when using VGGT, as it has many more model parameters (1B) compared to depth-anything-v2-small (24.8M). As a result, we continue to adopt the depth-anything-v2-small backbone. This experiment also demonstrates that our framework is not limited to a specific feature extractor.

5. Supplementary Video

We provide a supplementary video showcasing additional visual results rendered from multiple viewpoints. We highly recommend watching it to better appreciate the view consistency and high fidelity achieved by our method.

6. Limitations and Future Work

Although our proposed method demonstrates improvements in feed-forward 3D Gaussian Splatting for novel view synthesis, there are still certain limitations that could be addressed in future work. One key limitation of our model is that, although it is efficient, it may still face difficulties when processing very high-resolution datasets (e.g., those exceeding 1K), where fine details are not always accurately captured due to the inherent trade-off between model complexity and memory constraints. Moreover, our current Z-order transformer employs one or two Z-order blocks to aggregate Gaussian primitives. Using more such blocks can further reduce the number of Gaussian points; however, it leads to a noticeable degradation in performance, as illustrated in Fig. 7 of the main paper. Therefore, achieving higher compression without significant performance loss remains a challenging task.

In future work, several possible directions could be explored to further improve the proposed approach. One potential direction is to investigate hierarchical or multi-scale feature representations, which might help preserve fine-grained details when processing very high-resolution datasets while maintaining computational efficiency. It is also worth exploring alternative designs of the Z-order transformer, such as varying block configurations or introducing learnable aggregation depths to achieve a better balance between Gaussian primitive reduction and performance. Finally, combining the proposed framework with hybrid neural rendering methods [5, 14, 22, 34] might help enhance robustness and generalization across diverse scenarios.